

Week 3 Report

Implementing Version 2 of the App — Expanded MVP Functionality

Course: [Sum26] Practicum Project

Project: BackTestBench: modular backtesting and sandbox platform for trading strategies

Track: Industrial

TA: Gerald Imbugwa

Customer: Alexey Potemkin

Date: 23.06.2026

1. Team Information

- **Team name:** BackTestBench
- **Primary communication channel:** https://t.me/Muratich_29
- **Marat Diiarov** — Team lead, reports and customer communication, frontend.
- **Georgii Beliaev** — Tech lead, requirements and architecture, DevOps/QA.
- **Bogdan Zinchenko** — Data engineering and historical data management.
- **Samy Badavi** — Simulation Engine development.
- **Maxim Kharlamov** — Analytics and ranking.
- **Philip Oghenerukevwe Idiare** — Strategy Module.
- **Nikolay Taran** — Broker Adapter and T-Bank integration.

2. Project and Track Statement

BackTestBench is an industrial-track project that provides a modular environment for testing trading strategies on historical Russian market data. It aims to combine market-data access, configurable strategies, reproducible candle-by-candle simulation, performance analytics, and eventually strategy comparison and sandbox validation.

The Week 2 baseline consisted mainly of architecture, contracts, schemas, and planned API documentation. By the end of Week 3, the repository contained a working integrated path:

T-Bank data → MA Crossover strategy → Simulation Engine → TradeLog/equity curve → analytics → dashboard

The system is still an MVP. The demonstrated flow is limited to one strategy and mostly hardcoded run inputs, but it is now executable and measurable.

3. Week 3 Goal and Scope

Goal

Build the first end-to-end product version on top of the Week 2 architecture and make the core user journey demonstrable through a frontend.

Completed scope

- real historical candle retrieval from T-Bank;
- one configurable strategy implementation with validation;
- candle-by-candle long-only simulation;
- TradeLog, portfolio state, and equity-curve generation;
- core performance metrics;
- an integrated dashboard with pipeline status and a Run Backtest action;
- Docker/Compose support and PR smoke-build CI;
- backend unit and integration coverage.

Not completed in Week 3

- user-controlled strategy, instrument, timeframe, date range, and initial capital;
- multiple strategy comparison or parameter-grid experiments;
- buy/sell markers on the chart;
- percentage return and risk/reward metrics;
- relational persistence of runs, trades, and metrics;
- the planned FastAPI endpoint set;
- scheduler, notifications, trading bot, and automated Top-N recalculation.

4. Feature Expansion

4.1 Core Backtest Engine

PR [#82](#) introduced the Simulation Engine, portfolio model, execution context, BUY/SELL/HOLD semantics, order execution, and engine tests. The engine processes candles in chronological order, gives the strategy the current candle plus historical context, updates portfolio equity, and records closed trades.

PR [#89](#) extended the engine with an equity curve, a structured TradeLog, run context, trade timestamps, and a final-position close rule. An open long position is closed on the last candle so realized P&L remains consistent with final portfolio value minus initial capital.

4.2 Strategy Plugin and Configuration Flow

PR [#83](#) added:

- `BaseStrategy`;
- strategy registry and factory;
- YAML configuration parsing;
- validation of strategy parameters;
- the first built-in `ma_crossover` strategy;
- engine/strategy integration tests.

The MA Crossover strategy uses fast and slow simple moving averages. It emits BUY when the fast average crosses above the slow average, SELL on a downward crossover while holding a position, and HOLD during warm-up or when no action is required. Invalid combinations such as `fast >= slow`, non-integer windows, or a non-positive order size fail before simulation.

This extends the Week 2 skeleton by making the strategy boundary executable and additive: new strategies can be registered without changing the Simulation Engine.

4.3 Real Market Data

PR [#88](#) implemented the first T-Bank Broker Adapter integration. It:

- resolves a ticker such as SBER to the broker instrument identifier;
- requests real historical candles through the T-Bank Invest API;
- converts broker quotations and timestamps into the engine candle model;
- supports several timeframes;
- raises explicit authentication, rate-limit, invalid-instrument, and broker errors;
- reads `TINKOFF_TOKEN` from the environment rather than embedding it in source code.

The current integrated demo requests approximately 30 days of SBER hourly candles. Large ranges and small timeframes are not yet split into multiple requests or cached.

4.4 Analytics and Ranking Foundation

PR [#89](#) added:

- total realized P&L;
- annualized Sharpe ratio;
- maximum drawdown;
- win rate;
- a compounded 13% annual deposit baseline;
- data-integrity validation;
- a basic `build_top_n` helper that filters results below the deposit baseline.

The metrics handle empty inputs and zero-variance curves with neutral values rather than NaN. The ranking helper exists in code, but the current UI does not yet run multiple strategies or display a

Top-N list.

4.5 Dashboard and User Flow

PR [#90](#) integrated the Next.js dashboard with the Python pipeline. The implemented user journey is:

1. Open the dashboard.
2. Press **Run backtest**.
3. The frontend route creates a run ID and starts `main.py`.
4. The UI polls the dashboard API.
5. Pipeline stages change from pending/running to done or error.
6. The completed run displays P&L, Sharpe ratio, drawdown, win rate, deposit baseline, trade count, final portfolio state, and an equity chart.

This is a substantial extension over Week 2 because the product can now be demonstrated without running individual Python modules manually. The remaining limitation is that the button launches one predefined experiment: MA Crossover on SBER, 1-hour candles, 30-day history, and 100,000 RUB initial capital.

5. Integration and Data

Frontend/backend integration

The working integration uses a lightweight file-backed bridge:

POST `/api/run` → spawn `main.py` → update `data/runtime-dashboard.json`
→ GET `/api/dashboard` → React dashboard

`main.py` orchestrates the Broker Adapter, Strategy Module, Simulation Engine, and Analytics Module. It writes state atomically through a temporary file followed by `os.replace`, reducing the chance that the polling UI reads partially written JSON.

Data storage

The integrated Week 3 flow persists the latest dashboard state in `data/runtime-dashboard.json`. The repository contains an empty database package and a documented target relational schema, but the end-to-end flow does not yet persist each run, trade, and metric record into SQLite/PostgreSQL.

Issue [#40](#) is closed as “Store Metrics in Database,” but current code evidence supports file-backed runtime state, not completed relational metrics persistence. This discrepancy is recorded as a backlog and board-maintenance issue.

Error handling and validation

Implemented safeguards include:

- missing `TINKOFF_TOKEN` returns a failed start response;
- broker authentication, rate-limit, connection, and invalid-instrument errors;

- unsupported timeframe rejection;
- missing or malformed strategy configuration errors;
- strategy-specific parameter validation;
- empty broker response rejection in `main.py`;
- pipeline stages switch to error and preserve an error message;
- analytics consistency checks compare realized P&L with final portfolio value.

Remaining work includes request validation for user-selected inputs, broker retries, pagination/caching for large candle ranges, durable run state, and clearer user-facing recovery actions.

6. Updated API Documentation

Implemented Week 3 dashboard routes

POST /api/run

Starts the predefined backtest asynchronously.

- **Request body:** none.
- **Success:** 202 Accepted

```
{
  "ok": true,
  "started": true,
  "run_id": "uuid",
  "message": "Pipeline started"
}
```

- **Failure:** 500 when the token is missing or the process cannot be launched.

GET /api/dashboard

Returns the latest merged dashboard state with:

- run, strategy, instrument, timeframe, and data-source metadata;
- pipeline-stage statuses;
- P&L, Sharpe ratio, drawdown, win rate, and deposit baseline;
- equity points and trade count;
- final cash, position size, and equity;
- message, error, and last-update timestamp.

The route returns a valid empty/idle structure with status 200 if the runtime file cannot be read.

Planned API contract

[docs/api_description.md](#) documents the planned FastAPI routes such as

/backtest/run, /backtest/{run_id}, and /strategies. These routes remain target architecture and were not the integration path demonstrated in Week 3. The documentation must be reconciled with the Next.js route handlers or implemented behind FastAPI in the next iteration.

7. Industrial Track Contribution and Measurements

Expanded contribution

The project moved from an architecture baseline to a working product slice that integrates an external industrial API and multiple internal modules. The main contribution is a reproducible execution core around T-Bank historical data rather than a standalone strategy script.

Infrastructure work included:

- Docker images and Compose profiles in PR [#90](#);
- self-hosted PR build and smoke-check workflow in PR [#91](#);
- Docker port/profile corrections in PR [#92](#);
- removal of duplicated Compose services in PR [#93](#).

Updated measurements against the baseline

The Week 2 baseline had no executable measurement. A dashboard snapshot captured on June 23, 2026 showed one SBER/1h MA Crossover run with:

- initial capital: **100,000 RUB**;
- final equity: **100,028.67 RUB**;
- realized P&L: **+28.67 RUB** (approximately **+0.029%**, calculated for report context);
- Sharpe ratio: **0.042**;
- maximum drawdown: **2.49%**;
- win rate: **50%**;
- completed trades: **20**;
- deposit baseline for the same configured period: **1,009.59 RUB**.

The strategy therefore underperformed the deposit baseline in this demonstration run. The result is not a stable product benchmark: T-Bank data and the selected date window change, and a prior committed sample run had a negative result. The useful Week 3 achievement is that the platform can now produce comparable measurements and expose poor performance, not that the initial strategy is profitable.

Verification

- Backend: **30 tests passed** on June 23 (pytest -q).
- Frontend: Next.js 16.2.9 production build completed successfully.
- Current built routes: /, /api/dashboard, /api/run.

- CI: PR #91 configures Docker Compose build/run smoke checks on the self-hosted runner.

Non-blocking quality findings:

- pytest reported an upcoming `pytest-asyncio` fixture-loop-scope default change;
- Next.js warned about workspace-root inference from multiple lockfiles;
- Turbopack warned that dynamic filesystem tracing may include too much of the project;
- frontend lint fails on `react-hooks/set-state-in-effect` at `app/page.tsx:227`;
- `npm ci` reported two moderate dependency vulnerabilities.

8. Screenshots / Demo Evidence

Desktop dashboard

The screenshot shows the completed four-stage pipeline, current run metadata, five metric cards, trade count, and portfolio chart.

9. Internal Review and Customer Feedback

Review date

The expanded MVP was demonstrated to the customer on **22.06.2026**.

Acceptance review

The review confirmed that the core acceptance path works:

- the application is connected to real T-Bank market data;
- one strategy can run end to end;
- trades, final portfolio value, and analytics are produced;
- the frontend can launch and display a run.

The review also showed that the product is not yet a generally usable strategy-research tool because the experiment inputs are hardcoded.

Main issues and bugs found

1. Absolute P&L lacks sufficient context unless initial capital and percentage return are shown.
2. Win rate alone is not enough; risk/reward and return-based metrics are required.
3. Strategy parameters, instrument, timeframe, date range, and capital cannot be selected in the UI.
4. The portfolio chart lacks clear time labels and buy/sell markers.

5. Only one strategy and one parameter combination are demonstrated.
6. Large date ranges on small timeframes may exceed T-Bank request limits.
7. Historical data is downloaded as part of the run instead of being cached and reused.
8. Repeated BUY/SELL behavior and future stop-loss/take-profit ownership need an explicit contract.
9. Database/API documentation and the implemented runtime path are not fully aligned.
10. Long-running multi-strategy recalculation will require batch execution and progress tracking.
11. The narrow viewport has horizontal overflow/clipping in parts of the dashboard.

10. Updated Backlog

Next priority

- Add a backtest form for strategy, fast/slow windows, order size, instrument, timeframe, date range, and initial capital.
- Display initial capital, final capital, absolute P&L, and percentage return together.
- Add price/equity visualization with explicit entry and exit markers.
- Fix narrow-viewport overflow and verify the dashboard at common mobile breakpoints.
- Persist every run and TradeLog, then expose durable run history.
- Split large T-Bank candle intervals into multiple requests and cache normalized candles.
- Run several MA parameter combinations and rank comparable results.

Following priority

- Add at least one second strategy, preferably MA + RSI or MACD.
- Connect and display the existing Top-N ranking foundation.
- Define a ranking formula using return, Sharpe ratio, drawdown, and/or risk/reward.
- Reconcile the Next.js dashboard routes with the planned FastAPI API.
- Define repeated-signal, position-sizing, stop-loss, and take-profit semantics.
- Add background jobs for periodic data refresh and batch recalculation.

Lower priority

- live/sandbox trading bot;
- scheduler and notifications;
- multi-broker support;
- broader multi-asset support after the data/cache path is stable.

11. Required Links

Repository and planning

- Repository: <https://github.com/BackTest-bench-team/BackTestBench>
- Kanban board: <https://github.com/orgs/BackTest-bench-team/projects/1>
- Issues: <https://github.com/BackTest-bench-team/BackTestBench/issues>

Relevant pull requests

- [#81 — Backtesting architecture documentation](#)
- [#82 — Core backtest engine](#)
- [#83 — Strategy registry, config, MA Crossover, integration](#)
- [#88 — First T-Bank broker integration](#)
- [#89 — MVP-1 analytics metrics pipeline](#)
- [#90 — Frontend integration and Docker setup](#)
- [#91 — Self-hosted PR build pipeline](#)
- [#92 — Docker port/profile fix](#)
- [#93 — Remove duplicated Docker services](#)

Completed implementation issues

- [#17 — Basic metric calculations](#)
- [#37 — Strategy registry and factory](#)
- [#38 — Strategy configuration parser](#)
- [#42 — Minimal frontend skeleton](#)
- [#43 — First built-in strategy](#)
- [#44 — Strategy parameter validation](#)

Documentation and evidence

- Planned API documentation: https://github.com/BackTest-bench-team/BackTestBench/blob/main/docs/api_description.md
- Database schema: https://github.com/BackTest-bench-team/BackTestBench/blob/main/docs/database_schema.md
- Customer review notes: `transcriptions/22-06-26-customer.txt`
- Desktop screenshot: `reports/assets/week3-dashboard.png`
- Mobile screenshot: `reports/assets/week3-dashboard-mobile.png`

Conclusion

Week 3 delivered the first working BackTestBench product slice. The team integrated real T-Bank candles, a validated MA Crossover plugin, deterministic trade simulation, analytics, a launchable dashboard, Docker packaging, and PR smoke-build CI. The product now exposes measurable strategy behavior and can demonstrate when a strategy fails to beat its baseline.

The internal/customer review established the next direction: replace the single hardcoded experiment with configurable and comparable runs, improve result context and trade visualization, persist run history, and make market data reusable. These items form the practical Version 2 backlog.